MobiPredict — Movilidad urbana del Valle de Aburrá con datos reales

Flujo MLOps de extremo a extremo que pronostica la **velocidad de los corredores viales de Medellín a 1 hora**, emite **alertas tempranas de colapso** y caracteriza la **demanda y la densidad de viajes** con la **Encuesta Origen-Destino 2025 (AMVA)**. Todo con datos abiertos y reales; no hay datos simulados.

```
Fuentes oficiales → Estandarización → EDA → Features + split temporal → Entrenamiento + Optuna → MLflow → Selección → API + Frontend
(data/raw)          (data/interim)     (eda/)   (data/processed)            (RandomizedSearchCV)       (mlruns/)
(models/, docs/) (FastAPI, :8000)
```

> ¿Primera vez? Empieza por **`LEEME_PRIMERO.md`**: guía simple con todos los pasos para ejecutarlo en Windows.
>
> Informe técnico completo (cómo funciona cada etapa, cómo se construyeron los datos, decisiones y limitaciones): **`README_DETALLADO.md`**.

Orquestado con **Prefect** y ejecutable con un comando:

```
python run_pipeline.py
```

Fuentes de datos

Uso	Fuente	Cobertura
Serie del modelo	Velocidad y tiempo de viaje GT — SIMM, Secretaría de Movilidad de Medellín (MEData)	22 tramos de 11 corredores, cada hora 6:00-20:00, feb-2017 a dic-2020 (682.502 filas)
Demanda y densidad	EOD 2025 — API del visor oficial (AMVA)	6.490.299 viajes en día hábil, viajes por hora y modo, reparto modal, motorización, matriz OD de 67 macrozonas
Siniestralidad	Mendeley Data doi:10.17632/r6g5dfnpgh.1 (CC BY 4.0)	702.540 incidentes, 2008 a ago-2025
Lluvia	Open-Meteo Historical Weather (ERA5)	Horaria 2017-2020
Composición vehicular	Aforos vehiculares (MEData)	2017-2019, ~40 % motos
Festivos	Paquete `holidays` (Colombia)	2017-2026
Cifras de referencia	276 muertes viales en 2025; +48 % motos, −14 % TP (2017-2025)	2025

- Catálogo completo con URLs de descarga, licencias y tamaños: `data/fuentes_datos.csv`.
- Problemas de calidad encontrados y su tratamiento: `docs/fuentes_de_datos.md`.
- Ya incluidos en el repositorio: los agregados oficiales de la EOD 2025 (`data/reference/eod2025/`, incluida la matriz OD de 3.126 pares que suma 6.490.299 viajes), el diccionario y los centroides de las macrozonas, y la ubicación de las cámaras del SIMM.

Inicio rápido

Requisitos: Python 3.11-3.13 (probado con 3.12) y conexión a internet para la primera descarga (~175 MB).

```
py -3.12 -m venv .venv             # Windows   (Linux/Mac: python3 -m venv .venv)
source .venv/Scripts/activate      # Git Bash  (PowerShell: .venv\Scripts\Activate.ps1 · Linux/Mac: source
.venv/bin/activate)
pip install -r requirements-dev.txt

# 1) Descargar las fuentes reales a data/raw/ (cualquiera de las dos)
python scripts/descargar_datos.py
powershell -ExecutionPolicy Bypass -File scripts\descargar_datos.ps1

# 2) Pipeline completo: estandarización, EDA, features, entrenamiento, MLflow, selección
python run_pipeline.py             # completo (~15-25 min)
python run_pipeline.py --fast      # menos trials de búsqueda (~5 min)

# 3) Frontend + API y seguimiento
uvicorn api.main:app --port 8000                                                    # frontend:
http://localhost:8000  ·  API: /docs
set MLFLOW_ALLOW_FILE_STORE=true && mlflow ui --backend-store-uri ./mlruns --port 5000   # http://localhost:5000
```

`run_pipeline.py` descarga solo lo que falte en `data/raw/`. Si la API de la EOD no responde, usa la copia oficial de `data/reference/eod2025/`. Si falta una fuente obligatoria (velocidad, clima), se detiene con un mensaje claro: nunca inventa datos.

Estructura

```
mobipredict/
├── run_pipeline.py                # Punto de entrada (flujo Prefect)
├── scripts/
│   ├── descargar_datos.py         # Descarga todas las fuentes (multiplataforma)
│   ├── descargar_datos.ps1        # Igual, en PowerShell (Windows)
│   └── build_notebook.py          # Regenera el notebook desde el código
├── src/mobipredict/
│   ├── config.py                  # URLs oficiales, rutas, ventanas temporales, parámetros
│   ├── data_acquisition.py        # Descarga con reintentos + manifest.json (origen, tamaño, hash)
│   ├── fuentes.py                 # Lectura, validación y estandarización de cada fuente
│   ├── eda.py                     # 9 figuras y reportes con datos reales
│   ├── features.py                # Features espaciotemporales + EOD, split temporal
│   ├── training.py                # Candidatos, RandomizedSearchCV/Optuna, MLflow
│   ├── selection.py               # Reglas de selección, comparación, decisión
│   └── pipeline.py                # @flow y @task de Prefect
├── api/                           # FastAPI: /health, /tramos, /predict, /escenarios, /modelos…
├── frontend/                      # Página web (HTML/CSS/JS sin dependencias) servida en /
├── tests/                         # Pruebas con un extracto real (fixtures/)
├── notebooks/MobiPredict_MLOps_end_to_end.ipynb
├── data/
│   ├── fuentes_datos.csv          # Catálogo de fuentes y enlaces
│   ├── reference/                 # Tablas oficiales pequeñas (versionadas)
│   ├── raw/                       # Descargas (no versionadas)
│   ├── interim/                   # Tablas estandarizadas
│   └── processed/                 # train / valid / test / drift .parquet
├── eda/figures/  eda/reports/
├── models/                        # Ganadores, feature store, model_comparison.csv
├── docs/                          # model_decision.md, fuentes_de_datos.md
├── mlruns/                        # Tracking local de MLflow
└── Dockerfile  docker-compose.yml  Makefile  requirements*.txt
```

Componentes

1. Adquisición (`data_acquisition.py`, `fuentes.py`)

Cada fuente se toma de `data/raw/` si ya existe; si no, se descarga de la URL oficial con reintentos. El resultado queda en `data/raw/manifest.json` con origen, URL, tamaño y hash. `fuentes.py` valida y estandariza:

- **Velocidad:** la fecha sale de `FECHA_HORA`, porque las columnas `DÍA` y `MES` están corridas en parte de 2020. Las lecturas cada 30 minutos de 2020 y los 9.699 duplicados se promedian por hora. Los valores fuera de 2-110 km/h se descartan.
- **Tramos:** corredor + sentido. Cada tramo tiene longitud, tipo de flujo, ubicación (cámaras del SIMM) y macrozona EOD más cercana.
- **EOD 2025:** distribución horaria por grupo modal y matriz OD por macrozona.
- **Siniestros:** se lee el XLSX (hoja `BD`) y se guarda en parquet.

2. EDA (`eda.py`)

Figura	Hallazgo
`01_congestion_cronica_heatmap`	Congestión por tramo y hora en días laborables, pre-pandemia
`02_eod2025_viajes_por_hora`	6,49 M de viajes; pico de salida a las 6 h
`03_eod2025_reparto_modal`	A pie 32,7 %; la moto (14,6 %) supera al auto (11,7 %); 120,5 motos por 1.000 hab.
`04_eod2025_densidad_destinos`	La Candelaria y El Poblado atraen más viajes
`05_siniestralidad_vial`	Incidentes fatales por año 2008-2025 y por hora; cierre 2025: 276 muertes
`06_horas_en_colapso_por_tramo`	% de horas con velocidad < 50 % de la referencia
`07_drift_pandemia`	Caída de la congestión desde el 20-mar-2020
`08_lluvia_vs_colapso`	Más colapso en hora pico con lluvia fuerte
`09_composicion_vehicular_aforos`	~40 % de los vehículos aforados son motos

3. Features y split (`features.py`)

- **Tiempo:** hora, día y mes cíclicos; fin de semana, pico, festivo.
- **Rezagos:** velocidad y razón vel/vel-referencia en t-1, t-2, t-3 h, media de 3 h y tendencia. Los rezagos no cruzan la noche.
- **Clima:** lluvia actual y de las últimas 3 h.
- **Espacio:** longitud, tipo de flujo, ubicación y velocidad de referencia (p95 del entrenamiento) del tramo.
- **Perfil histórico:** velocidad media del tramo por tipo de día y hora objetivo, calculada solo con entrenamiento.
- **EOD 2025:** participación horaria de viajes privados, públicos y totales en la hora objetivo, y viajes generados y atraídos por la macrozona del tramo.
- **Targets:** velocidad en t+1 h (regresión) y colapso en t+1 h = velocidad < 50 % de la referencia (clasificación).

- **Split temporal:** entrenamiento < 2019-09-01 ≤ validación < 2019-12-01 ≤ prueba < 2020-03-20 ≤ drift (pandemia) < 2021.

4-6. Entrenamiento, optimización y MLflow (`training.py`)

Tarea	Línea base	Candidatos	Búsqueda
Velocidad a 1 h	Perfil histórico	Ridge, Random Forest, LightGBM, red neuronal MLP	RandomizedSearchCV / Optuna con `TimeSeriesSplit`
Colapso a 1 h	Regla de velocidad	Regresión logística, Random Forest, LightGBM	RandomizedSearchCV / Optuna (`average_precision`)

Cada modelo es un run anidado en MLflow con parámetros, métricas `valid_*`, `test_*` y `drift_*` (MAE, RMSE, WAPE, R^2, precisión, recall, F1, PR-AUC, Brier, falsas alarmas por día), importancias y el modelo. Los ganadores se registran como `mobipredict-velocidad` y `mobipredict-colapso` con alias `champion`.

7. Selección (`selection.py`)

Reglas fijadas antes de ver resultados y aplicadas en validación:

- **Velocidad:** debe mejorar ≥ 10 % el WAPE del perfil histórico. Gana el menor RMSE; si otro queda a menos de 1 %, gana el más rápido.
- **Colapso:** solo modelos con recall ≥ 0,70. Gana la mayor PR-AUC; si empatan (< 0,005), el de menor Brier.

Salidas: `models/model_comparison.csv` y `docs/model_decision.md`. Este último incluye la prueba de estrés de la pandemia.

API

Método	Ruta	Descripción
GET	`/health`	Estado y modelos cargados
GET	`/tramos`	Los 22 tramos con su `tramo_id`
POST	`/predict`	Velocidad y tiempo de viaje a 1 h, probabilidad y alerta de colapso
POST	`/predict/batch`	Hasta 500 peticiones
GET	`/escenarios`	Momentos reales del periodo de prueba (y uno de la cuarentena)
GET	`/escenarios/{id}`	Lecturas de cada tramo hasta la hora t y la velocidad medida en t + 1 h
GET	`/modelos`	Ganadores, razones de la selección y métricas de todos los candidatos
GET	`/figuras/{archivo}`	Figuras del EDA
POST	`/admin/reload`	Recarga modelos tras reentrenar

```
curl -X POST http://localhost:8000/predict -H "Content-Type: application/json" -d '{
  "tramo_id": "AVENIDA_80_NS",
  "observaciones": [
    {"timestamp": "2020-02-11T14:00:00", "velocidad_kmh": 24.0, "lluvia_mm": 0.0},
    {"timestamp": "2020-02-11T15:00:00", "velocidad_kmh": 22.0, "lluvia_mm": 0.0},
    {"timestamp": "2020-02-11T16:00:00", "velocidad_kmh": 19.0, "lluvia_mm": 0.4},
    {"timestamp": "2020-02-11T17:00:00", "velocidad_kmh": 15.0, "lluvia_mm": 1.2}
  ]}'
```

La respuesta trae `velocidad_predicha_kmh`, `tiempo_viaje_predicho_min`, `probabilidad_colapso`, `alerta_colapso` y `nivel` (`normal`, `precaucion` o `colapso`).

Frontend

Al encender la API, **http://localhost:8000** abre la aplicación web (no requiere instalar nada más):

- **Predicción:** eliges un momento real del periodo de prueba (mediodía, pico de la tarde, tarde con lluvia, la hora con más colapsos o la cuarentena de 2020). El modelo predice las 22 vías una hora adelante y lo compara con lo que de verdad se midió: alertas, aciertos y error medio.
- **Detalle por vía:** gráfico de las últimas horas, predicción y valor real, probabilidad de colapso frente al umbral, y un formulario para probar otros valores de velocidad o lluvia.
- **Modelos:** ganadores, razones de la selección y tabla comparativa de todos los candidatos (prueba y pandemia).
- **Análisis de datos:** los 9 gráficos del EDA.

Los momentos reales se generan en el pipeline (`models/escenarios.json`). Si no existen, el frontend pasa a modo manual.

Docker

```
python scripts/descargar_datos.py        # o el .ps1; los datos se montan como volumen
docker compose up --build                # trainer -> frontend + API (:8000) + MLflow UI (:5000)
docker compose --profile orquestacion up # + Prefect UI (:4200)
RETRAIN=1 docker compose up trainer      # reentrenar
```

Pruebas

```
pytest -q tests
```

- `test_features.py` corre sobre un extracto real (Av. 80 y Autopista Sur, 10-11 feb 2020). Verifica el parser de fechas, la agregación horaria, que los rezagos no crucen la noche, que no haya fuga en los targets y que las features de la API sean idénticas a las de entrenamiento.
- `test_fuentes.py` verifica que se lea el Excel de datos de siniestralidad (y no el diccionario que viene en el mismo ZIP).
- `test_api.py` prueba los endpoints, el frontend y los escenarios una vez entrenados los modelos.

Limitaciones

- La serie abierta de velocidad termina en dic-2020; la EOD es de 2025. El modelo aprende la dinámica de congestión 2017-2020 y usa la EOD como prior de demanda. Para operar hoy hay que reentrenar con el feed actual del SIMM: basta un CSV con las mismas columnas en `data/raw/`.
- Las velocidades vienen de tiempos de viaje de Google Traffic agregados por la Alcaldía, no de espiras.

- No hay conteos de flujo vehicular abiertos y continuos; por eso el modelo predice velocidad y tiempo de viaje en lugar de flujo.